ESP32 Upload Issue

This is a copy of a shared ChatGPT conversation

Today 12:54 AM

> Sketch uses 1072537 bytes (34%) of program storage space. Maximum is 3145728 bytes.
> Global variables use 69456 bytes (21%) of dynamic memory, leaving 258224 bytes for local variables. Maximum is 327680 bytes.
> esptool v5.3.0
> Serial port /dev/cu.usbserial-110:
> Connecting.....
>
> A fatal error occurred: Failed to connect to ESP32: No serial data received.
> For troubleshooting steps visit:
> https://docs.espressif.com/projects/esptool/en/latest/troubleshooting.html
> Failed uploading: uploading error: exit status 2
> Show moreShow less

তোমার code **compile ঠিকমতো হয়েছে**। সমস্যা code-এ না—ESP32 **upload/boot mode-এ ঢুকছে না** অথবা serial communication হচ্ছে না।

Error:

```
Failed to connect to ESP32: No serial data received.
```

মানে Mac `/dev/cu.usbserial-110` port পাচ্ছে, কিন্তু ESP32 থেকে response পাচ্ছে না।

প্রথমে এই methodটা করো

যদি তোমারটা সাধারণ **ESP32-WROOM-32 DevKit** হয়:

1. Arduino IDE → **Tools** → **Board** → **ESP32 Arduino** → **ESP32 Dev Module**
2. Tools → Port → `/dev/cu.usbserial-110`
3. **Upload Speed** → **115200**
4. Upload (→) চাপো।
5. নিচে যখন দেখাবে:
   `Connecting........`
   তখন ESP32-এর **BOOT button চেপে ধরে রাখো**।
6. BOOT ধরে রেখেই **EN/RESET button একবার press করে ছেড়ে দাও**।
7. BOOT এখনও ধরে রাখো।
8. Upload শুরু হয়ে `Writing at...` / percentage দেখালে **BOOT ছেড়ে দাও**।

সঠিক হলে এরকম আসবে:

```
Connecting...
Chip is ESP32...
```

```
Writing at 0x00010000...
Writing at...
100%
Hard resetting via RTS pin...
```

তাও না হলে GPIO0 দিয়ে manual upload mode

ESP32 power off/USB খুলে:

```
ESP32
GPIO0  -------- GND
```

তারপর USB connect করো এবং Upload দাও।

অর্থাৎ upload-এর সময়:

```
GPIO0 = GND
        └── ESP32 Download Mode
```

Upload শেষ হলে **GPIO0–GND connection খুলে** EN/RESET চাপবে।

যদি ESP32-CAM হয়

ESP32-CAM-এর ক্ষেত্রে connection আলাদা:

```
USB-TTL / Programmer     ESP32-CAM
---------------------------------
5V                    → 5V
GND                   → GND
TX                    → U0R
RX                    → U0T

GPIO0 ---------------- GND
```

খেয়াল রাখবে:

```
TX → RX
RX → TX
```

তারপর GPIO0→GND রেখে reset দিয়ে upload করবে। Upload শেষ হলে GPIO0→GND খুলে reset দেবে।

একটা গুরুত্বপূর্ণ বিষয়: তোমার log দেখে USB serial adapter detect হচ্ছে, তাই driver/port পুরোপুরি missing মনে হচ্ছে না। এখন সবচেয়ে সম্ভাব্য কারণ হলো **ESP32 download mode-এ না যাওয়া, TX/RX wiring সমস্যা (যদি external programmer ব্যবহার করো), অথবা power সমস্যা।**

Espressif-এর troubleshooting guide: ESP32 esptool troubleshooting

তুমি যদি **ESP32-WROOM-32 board-এর একটা clear ছবি** দাও এবং বলো **USB cable দিয়ে direct নাকি Arduino UNO/USB-TTL দিয়ে upload করছ**, আমি তোমাকে exact pin-to-pin connection দেখিয়ে দেব।

> ESP32-CAM-MB Type-C Download Module for ESP32 CAM
> modiul ta kine aneci

হ্যাঁ, তাহলে কাজটা অনেক সহজ। **ESP32-CAM-MB Type-C Download Module** থাকলে Arduino UNO বা আলাদা FTDI/USB-TTL লাগবে না। এই board-এ USB-to-Serial converter built-in থাকে এবং ESP32-CAM সরাসরি বসানো যায়।

Robotics BD Store+1

এখন এভাবে করো

ESP32-CAM-টা **CAM-MB board-এর socket-এ ঠিক orientation-এ** বসাও। তারপর:

```
MacBook
   │
   │ USB Type-C cable
   ▼
ESP32-CAM-MB
   │
   ▼
ESP32-CAM
```

Arduino IDE-তে:

```
Tools → Board → ESP32 Arduino
      → AI Thinker ESP32-CAM

Tools → Port
      → /dev/cu.usbserial-110

Upload Speed
      → 115200
```

তারপর **Upload** চাপো।

যদি আবার:

```
Connecting.................
Failed to connect to ESP32
```

আসে, তাহলে ESP32-CAM-MB-এর **IO0/BOOT/Download button** থাকলে সেটা চেপে ধরে **RST একবার press করে ছেড়ে দাও**, তারপর BOOT/IO0 ছাড়ো। এই sequence ESP32-CAM-কে download mode-এ ঢোকায়। Manuals++1

গুরুত্বপূর্ণ

তোমার আগের error অনুযায়ী Mac ইতিমধ্যে:

```
Serial port /dev/cu.usbserial-110
```

detect করছে। তাই নতুন CAM-MB লাগানোর পর **Port আবার check করবে**—port নাম বদলে যেতে পারে।

আর অবশ্যই **data-capable Type-C cable** ব্যবহার করবে; শুধু charging cable হলে upload হবে না।

এখন তোমার ESP32-CAM + নতুন ESP32-CAM-MB একসাথে লাগানো অবস্থায় একটা clear ছবি দাও। আমি দেখে বলে দিতে পারব orientation ঠিক আছে কিনা এবং কোন button কখন চাপতে হবে।

> Sketch uses 1072537 bytes (34%) of program storage space. Maximum is 3145728 bytes.
> Global variables use 69456 bytes (21%) of dynamic memory, leaving 258224 bytes for local variables. Maximum is 327680 bytes.
> esptool v5.3.0
> Serial port /dev/cu.usbserial-110:
> Connecting.....

A fatal error occurred: Failed to connect to ESP32: No serial data received.
For troubleshooting steps visit:
https://docs.espressif.com/projects/esptool/en/latest/troubleshooting.html
Failed uploading: uploading error: exit status 2
Show moreShow less

হ্যাঁ—এখনও একই সমস্যা। **Code compile হচ্ছে, port-ও detect হচ্ছে, কিন্তু ESP32-CAM download/flash mode-এ ঢুকছে না অথবা CAM-MB ↔ ESP32-CAM serial connection হচ্ছে না।**

ESP32-CAM-MB থাকলে আগে এই exact procedure করো।

১. Arduino IDE settings

```
Board: AI Thinker ESP32-CAM
Port: /dev/cu.usbserial-110
Upload Speed: 115200
Flash Frequency: 40MHz
Flash Mode: QIO
Partition Scheme: Huge APP (3MB No OTA/1MB SPIFFS)
```

২. Upload-এর সময় এই sequence

ESP32-CAM-MB Type-C দিয়ে Mac-এ connected রাখো।

Arduino IDE-তে **Upload** চাপো।

যখন:

```
Connecting.....
```

দেখাবে, তখন ESP32-CAM-এর **RST button একবার চাপো।**

যদি না হয়, manual flash mode try করো:

```
IO0 → GND
```

তারপর:

1. **`IO0`** এবং **`GND`** short করো
2. **`RST`** একবার press করো
3. Arduino IDE → Upload
4. Upload শেষ হওয়া পর্যন্ত IO0→GND রাখো
5. Upload successful হলে IO0→GND খুলে ফেলো
6. আবার RST চাপো

Successful হলে এরকম আসবে:

```
Connecting....
Chip is ESP32...
Writing at 0x00001000...
Writing at 0x00010000...
...
```

```
100%
Hard resetting via RTS pin...
```

৩. খুব গুরুত্বপূর্ণ test

Arduino IDE → **Tools** → **Serial Monitor** খোলো।

Baud rate:

```
115200
```

তারপর ESP32-CAM-এর **RST** button চাপো।

দেখো Serial Monitor-এ কিছু আসে কিনা, যেমন:

```
rst:0x1 (POWERON_RESET)
boot:0x13 (SPI_FAST_FLASH_BOOT)
...
```

অথবা

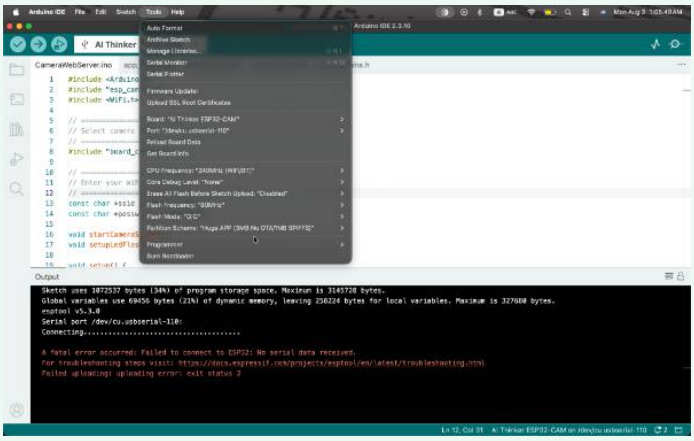
```
waiting for download
```

যদি RST চাপার পর Serial Monitor একদম blank থাকে, তাহলে সমস্যাটা code/Arduino IDE-এর হওয়ার সম্ভাবনা কম। তখন সম্ভাব্য কারণ:

- ESP32-CAM CAM-MB-তে উল্টো/ভুলভাবে বসানো
- ESP32-CAM board faulty
- CAM-MB board faulty
- socket/pin contact হচ্ছে না
- ESP32-CAM power পাচ্ছে না
- USB cable/USB serial communication সমস্যা

Espressif-ও `No serial data received`-কে মূলত serial communication না পাওয়ার সমস্যা হিসেবে ব্যাখ্যা করে। Espressif esptool troubleshooting

এখন সবচেয়ে useful কাজ: ESP32-CAM-টা CAM-MB module-এ লাগানো অবস্থায় **উপর থেকে একটা clear photo পাঠাও**, যেন ESP32-CAM, CAM-MB, Type-C port এবং pins দেখা যায়। আমি ছবিতে দেখে connection/orientation ঠিক আছে কিনা বলে দিতে পারব।



ছবিটা দেখে Arduino IDE-এর configuration **ঠিক আছে**:

- Board: **`AI Thinker ESP32-CAM`** ✅
- Port: **`/dev/cu.usbserial-110`** ✅
- CPU: **`240MHz`** ✅
- Flash Frequency: **`80MHz`** — আপাতত সমস্যা না
- Flash Mode: **`QIO`** ✅
- Partition: **`Huge APP (3MB No OTA/1MB SPIFFS)`** ✅
- Code compile হচ্ছে ✅

তাই এখন মূল সমস্যা **ESP32-CAM flash/download mode বা hardware communication**।

এখন এই test-টা করো

১. **Tools → Serial Monitor** খোলো।

Baud rate **115200** করো।

তারপর ESP32-CAM-MB-তে ESP32-CAM লাগানো অবস্থায় **RST/RESET button একবার চাপো**।

Serial Monitor-এ কী আসে সেটা আমাকে screenshot দাও।

যদি কিছুই না আসে, সেটাও বলো: **"blank"**।

তারপর manual upload try করবে

যদি তোমার ESP32-CAM-MB-তে **`BOOT`** button থাকে:

BOOT চেপে ধরে → RST একবার চাপো → RST ছাড়ো → BOOT ছাড়ো → Upload চাপো।

আর যদি শুধু **`RST`** থাকে, ESP32-CAM-এর:

```
IO0 ——— GND
```

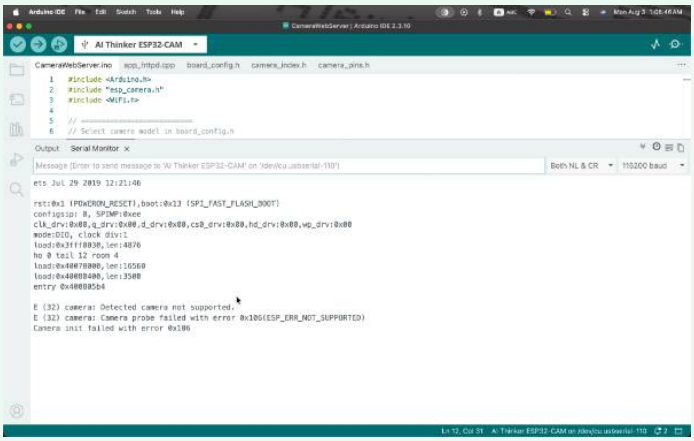
jumper wire দিয়ে connect করে RST চাপো, তারপর Upload দাও।

এখন ideally:

```
Connecting...
Chip is ESP32...
```

আসা উচিত।

একটা জিনিস এখন সবচেয়ে দরকার: ESP32-CAM-টা **ESP32-CAM-MB Type-C module-এর উপর লাগানো অবস্থার একটা ছবি পাঠাও**। Arduino IDE-এর আর screenshot দরকার নেই। Board-এর দুই পাশের pin এবং Type-C connector দেখা গেলে আমি orientation/wiring-এ সমস্যা আছে কি না দেখে বলতে পারব।



এটা খুব useful result। এখন বোঝা যাচ্ছে **ESP32-CAM board alive এবং Serial communication কাজ করছে**। 🎉

Screenshot-এ:

```
rst:0x1 (POWERON_RESET)
boot:0x13 (SPI_FAST_FLASH_BOOT)
```

মানে ESP32 ঠিকমতো boot করছে।

কিন্তু এখন error:

```
E (32) camera: Detected camera not supported.
E (32) camera: Camera probe failed with error 0x106 (ESP_ERR_NOT_SUPPORTED)
Camera init failed with error 0x106
```

এটা **আগের upload error-এর থেকে আলাদা সমস্যা**। এখন ESP32 camera module-কে ঠিকমতো detect করতে পারছে না।

প্রথমে camera ribbon cable check করো

USB খুলে power off করো। ESP32-CAM-এর camera ribbon cable connector থেকে carefully খুলে আবার ভালোভাবে লাগাও।

বিশেষ করে নিশ্চিত করো:

```
Camera
   │
   │ ribbon cable
   ▼
[ ESP32-CAM connector ]
        ↑
 সম্পূর্ণ ভিতরে ঢোকানো
```

Ribbon cable একটু loose হলেও ঠিক এই ধরনের:

```
Camera probe failed
ESP_ERR_NOT_SUPPORTED
```

আসতে পারে।

তারপর `board_config.h` check করো

তোমার screenshot-এ **`board_config.h`** tab আছে।

সেটা open করো এবং camera model section খুঁজে বের করো। সেখানে **AI Thinker** model selected থাকতে হবে।

নতুন CameraWebServer example-এ সাধারণত এরকম configuration থাকবে:

```
C++
#define CAMERA_MODEL_AI_THINKER
```

অন্য camera model যেন simultaneously enabled না থাকে।

যেমন AI Thinker ESP32-CAM হলে চাই:

```
C++
#define CAMERA_MODEL_AI_THINKER
```

তারপর আবার upload করতে হবে।

কিন্তু একটা গুরুত্বপূর্ণ বিষয়

তোমার Serial Monitor দেখে বোঝা যাচ্ছে ESP32 এখন:

```
SPI_FAST_FLASH_BOOT
```

mode-এ যাচ্ছে, অর্থাৎ **normal run mode**।

Upload করতে হলে boot log-এ download mode দরকার। তাই আগের:

```
Failed to connect to ESP32
```

সমস্যাটা এখনও আলাদাভাবে থাকতে পারে।

ESP32-CAM-MB দিয়ে upload করার সময় দরকার হলে:

```
IO0 → GND
      ↓
Press RST
      ↓
Upload
```

Successful connection হলে:

```
Connecting...
Chip is ESP32...
```

আসবে।

এখন `board_config.h` tab-এ click করে screenshot পাঠাও। সেটা দেখলে তোমার CameraWebServer code-এ সঠিক camera model selected আছে কি না আমি সরাসরি বলে দিতে পারব।

> eita esp32s cam

বুঝলাম—তোমারটা **ESP32S-CAM**। এখানে একটা গুরুত্বপূর্ণ বিষয় হলো: Arduino IDE-তে **`AI Thinker ESP32-CAM`** board নির্বাচন করা অনেক ESP32S-CAM-এর ক্ষেত্রেও ঠিক, কিন্তু **camera model/pin configuration** অবশ্যই তোমার module-এর camera hardware-এর সঙ্গে match করতে হবে।

তোমার Serial Monitor-এর এই অংশটা গুরুত্বপূর্ণ:

```
E (32) camera: Detected camera not supported.
E (32) camera: Camera probe failed with error 0x106 (ESP_ERR_NOT_SUPPORTED)
Camera init failed with error 0x106
```

এর মানে ESP32 নিজে চলছে এবং serial connection-ও চলছে, কিন্তু **camera sensor detect হচ্ছে না/unsupported sensor detect হচ্ছে**।

প্রথমে camera-র ছোট sensor board-এর গায়ে কী লেখা আছে দেখো। সাধারণ ESP32S-CAM-এ **OV2640** থাকার কথা। যদি **`OV2640`** লেখা থাকে, তাহলে CameraWebServer example-এ **`board_config.h`** খুলে **`CAMERA_MODEL_AI_THINKER`** selected রাখো:

```
C++
#define CAMERA_MODEL_AI_THINKER
```

অন্য camera model define করা থাকলে সেটা comment করে দাও।

তারপর **camera ribbon cable খুলে আবার শক্ত করে বসাও**। Power/USB disconnect করে এটা করবে। Ribbon cable উল্টো, loose বা পুরোপুরি socket-এর মধ্যে না ঢুকলে **`Camera probe failed`** আসতে পারে।

আর তোমার আগের upload সমস্যাটাও আমরা এখন narrow down করতে পেরেছি: Serial Monitor output আসছে মানে **ESP32S-CAM এবং CAM-MB-এর serial path কাজ করছে**। Upload-এর সময় শুধু board-টাকে download mode-এ নিতে হবে।

ESP32S-CAM board + camera-টার একটা clear ছবি পাঠাও, বিশেষ করে camera sensor-এর লেখা এবং ESP32S-CAM-এর সামনের দিক। আমি দেখে তোমার exact camera model এবং কোন **`#define CAMERA_MODEL_...`** ব্যবহার করতে হবে বলে দেব।

> camera rebon a GC2145
> lekha

হ্যাঁ—এটাই আসল clue। তোমার camera **OV2640 না, GC2145**।

ভালো খবর হলো, Espressif-এর বর্তমান **`esp32-camera`** driver-এ **GC2145 support আছে**। GitHub+1

কিন্তু GC2145-এর একটা গুরুত্বপূর্ণ পার্থক্য আছে: এটা **native JPEG output support করে না**। তাই standard **`CameraWebServer`** example-এর **`PIXFORMAT_JPEG`** configuration GC2145-এর জন্য সমস্যা করতে পারে। Driver নিজেও unsupported JPEG format হলে error দেয়। GitHub

তবে তোমার বর্তমান error:

```
Detected camera not supported.
Camera probe failed with error 0x106
```

আরও আগে হচ্ছে—sensor detection stage-এ। অর্থাৎ প্রথমে **GC2145 detect করাতে হবে**।

প্রথমে `board_config.h`-এ:

```
C++
#define CAMERA_MODEL_AI_THINKER
```

রাখো, কারণ এটা মূলত ESP32-CAM board-এর **GPIO pin mapping** ঠিক করবে। Camera sensor GC2145 হওয়ায় এরপর code-এ JPEG-এর বদলে:

```
C++
config.pixel_format = PIXFORMAT_RGB565;
```

ব্যবহার করা লাগতে পারে।

GC2145 নিয়ে কাজ করা একটি community example-এও `PIXFORMAT_RGB565`, কম XCLK এবং ছোট frame size ব্যবহার করে সফল হওয়ার কথা বলা হয়েছে। Reddit

আমার মনে হয় standard CameraWebServer code modify করতে করতে ঝামেলা করার চেয়ে তোমার **ESP32S-CAM + GC2145-এর জন্য একদম নতুন test code** দিয়ে শুরু করা ভালো।

চাইলে আমি এখন তোমাকে **GC2145 detect → ছবি capture → Serial Monitor-এ camera OK দেখাবে** এমন minimum test code দিতে পারি; সেটা কাজ করলে তারপর **Wi-Fi Web Server live camera** code বানাব।

> ok daw

ঠিক আছে। প্রথমে **GC2145 camera sensor detect হচ্ছে কি না** সেটা test করি। যেহেতু তোমার board ESP32S-CAM এবং camera ribbon-এ `GC2145` লেখা, নিচের minimal code upload করো।

GC2145 Camera Test Code

Arduino IDE-তে নতুন sketch খুলে পুরো codeটা paste করো:

```
C++
#include "esp_camera.h"

// ===========================================
// ESP32S-CAM / AI Thinker style PIN mapping
// ===========================================

#define PWDN_GPIO_NUM     32
#define RESET_GPIO_NUM    -1

#define XCLK_GPIO_NUM      0
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27

#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5

#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22


void setup() {
```

```
  Serial.begin(115200);
  delay(2000);

  Serial.println();
  Serial.println("============================");
  Serial.println(" ESP32S-CAM GC2145 TEST");
  Serial.println("============================");

  camera_config_t config;

  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer   = LEDC_TIMER_0;

  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;

  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;

  config.pin_sccb_sda = SIOD_GPIO_NUM;
  config.pin_sccb_scl = SIOC_GPIO_NUM;

  config.pin_pwdn  = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;

  // Start conservatively for GC2145
  config.xclk_freq_hz = 10000000;

  // GC2145 does not provide native JPEG like OV2640
  config.pixel_format = PIXFORMAT_RGB565;

  config.frame_size = FRAMESIZE_QVGA;

  config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
  config.fb_location = CAMERA_FB_IN_PSRAM;

  config.jpeg_quality = 12;
  config.fb_count = 1;


  Serial.println("Initializing camera...");

  esp_err_t err = esp_camera_init(&config);

  if (err != ESP_OK) {

    Serial.printf(
      "Camera initialization FAILED!\nError: 0x%x\n",
      err
    );

    Serial.println();
    Serial.println("Check:");
    Serial.println("1. Camera ribbon cable");
    Serial.println("2. GC2145 sensor");
    Serial.println("3. Camera pin configuration");

    return;
  }


  Serial.println();
  Serial.println("Camera initialized successfully!");
  Serial.println("----------------------------");
```

```
  // Get sensor information

  sensor_t *s = esp_camera_sensor_get();

  if (s != NULL) {

    Serial.println("Camera sensor detected.");

    Serial.printf(
      "Sensor PID: 0x%04X\n",
      s->id.PID
    );

  } else {

    Serial.println("Could not read sensor information.");

  }


  // Capture one frame

  Serial.println();
  Serial.println("Capturing test frame...");

  camera_fb_t *fb = esp_camera_fb_get();

  if (!fb) {

    Serial.println("Frame capture FAILED!");

    return;
  }


  Serial.println("Frame captured successfully!");

  Serial.printf(
    "Width: %d\n",
    fb->width
  );

  Serial.printf(
    "Height: %d\n",
    fb->height
  );

  Serial.printf(
    "Frame size: %u bytes\n",
    fb->len
  );


  // Release framebuffer

  esp_camera_fb_return(fb);


  Serial.println();
  Serial.println("============================");
  Serial.println(" GC2145 TEST COMPLETE");
  Serial.println("============================");
}


void loop() {

  // Nothing required here

}
```

Arduino IDE settings

তোমার বর্তমান settings মোটামুটি ঠিক আছে। রাখো:

```
Board: AI Thinker ESP32-CAM
Port: /dev/cu.usbserial-110

CPU Frequency: 240MHz
Flash Mode: QIO
Partition Scheme: Huge APP
```

Upload speed যদি option পাও:

```
115200
```

দাও।

Upload করার পরে

Upload successful হলে **Serial Monitor → 115200 baud** করো এবং **RST** চাপো।

কাজ করলে এমন কিছু পাওয়ার কথা:

```
============================
 ESP32S-CAM GC2145 TEST
============================

Initializing camera...

Camera initialized successfully!
----------------------------
Camera sensor detected.
Sensor PID: 0x....

Capturing test frame...

Frame captured successfully!
Width: 320
Height: 240
Frame size: .... bytes

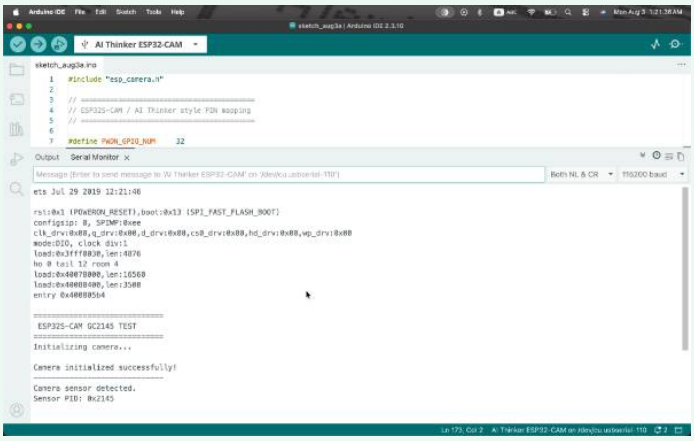
============================
 GC2145 TEST COMPLETE
============================
```

যদি আবার:

```
Detected camera not supported
Camera probe failed with error 0x106
```

আসে, **Serial Monitor-এর পুরো output screenshot পাঠাও**। তখন আমরা driver version এবং GC2145 detection check করব—কারণ ওই অবস্থায় শুধু code পরিবর্তন না করে তোমার installed ESP32 Arduino core-এর **esp32-camera** version-ও দেখা দরকার।

আর যদি **Camera initialized successfully!** আসে, তাহলে পরের ধাপে তোমার **GC2145-এর জন্য Wi-Fi Camera Web Server** বানিয়ে দেব, যাতে Mac/phone browser থেকে camera image দেখা যায়।



দারুণ—এবার **GC2145 camera successfully detect হয়েছে**। Screenshot-এ পরিষ্কার:

```
ESP32S-CAM GC2145 TEST
Initializing camera...

Camera initialized successfully!

Camera sensor detected.
Sensor PID: 0x2145
```

`PID: 0x2145` নিশ্চিত করছে sensor হচ্ছে **GC2145**। অর্থাৎ ESP32S-CAM, camera ribbon connection, pin mapping এবং camera driver—সব কাজ করছে। ✅

এখন next step হলো **Wi-Fi Camera Web Server** বানানো, যাতে ESP32S-CAM Wi-Fi-তে connect হয়ে IP address দেয় এবং তুমি Mac/phone browser থেকে camera দেখতে পারো।

তোমার GC2145 native JPEG camera না হওয়ায় standard `CameraWebServer` example সরাসরি ব্যবহার না করে **GC2145-compatible web server code** করা ভালো।

চাইলে আমি এখন তোমাকে complete code দিতে পারি, যেখানে:

ESP32S-CAM + GC2145 → Wi-Fi → IP Address → Browser → Camera View

এবং চাইলে ESP32 নিজেই hotspot তৈরি করবে, তাই **router/internet-ও লাগবে না**।

> ok daw

অবশ্যই। যেহেতু তোমার **GC2145 এখন successfully detect হচ্ছে (`PID: 0x2145`)**, এবার ESP32S-CAM নিজেই Wi-Fi hotspot তৈরি করবে। Phone/Mac সেই Wi-Fi-তে connect করে browser থেকে camera image দেখতে পারবে।

GC2145 native JPEG দেয় না, তাই আমরা `RGB565` capture করে ESP32-তেই JPEG-এ convert করব।

ESP32S-CAM + GC2145 Wi-Fi Camera Server

পুরো আগের code delete করে এটা paste করো:

```C++
#include <WiFi.h>
#include <WebServer.h>
#include "esp_camera.h"
#include "img_converters.h"
```

```
// ================================================
//          ESP32S-CAM + GC2145 PIN CONFIGURATION
// ================================================

#define PWDN_GPIO_NUM     32
#define RESET_GPIO_NUM    -1

#define XCLK_GPIO_NUM      0
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27

#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5

#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22


// ================================================
//                 Wi-Fi Hotspot
// ================================================

const char* ap_ssid = "ESP32-GC2145-CAM";
const char* ap_password = "12345678";

WebServer server(80);


// ================================================
//                    WEB PAGE
// ================================================

const char INDEX_HTML[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>

<meta name="viewport" content="width=device-width, initial-scale=1">

<title>ESP32 GC2145 Camera</title>

<style>

body {
  background: #111;
  color: white;
  font-family: Arial;
  text-align: center;
  margin: 0;
  padding: 20px;
}

h1 {
  color: #00e5ff;
}

.card {
  max-width: 700px;
  margin: auto;
  background: #222;
  padding: 20px;
  border-radius: 15px;
}

img {
  width: 100%;
  max-width: 640px;
  border-radius: 10px;
```

```
  margin-top: 15px;
}

button {
  background: #00e5ff;
  border: none;
  padding: 12px 25px;
  margin: 10px;
  border-radius: 8px;
  font-size: 16px;
  cursor: pointer;
}

</style>

</head>

<body>

<div class="card">

<h1>ESP32S-CAM</h1>

<h3>GC2145 Camera</h3>

<p>Camera Live View</p>

<img id="camera" src="/capture">

<br>

<button onclick="refreshCamera()">
Refresh
</button>

</div>


<script>

function refreshCamera() {

  document.getElementById("camera").src =
  "/capture?t=" + new Date().getTime();

}


// Refresh image every 1 second

setInterval(refreshCamera, 1000);

</script>


</body>
</html>
)rawliteral";


// ====================================================
//                   HOME PAGE
// ====================================================

void handleRoot() {

  server.send_P(
    200,
    "text/html",
    INDEX_HTML
  );

}
```

```
// ========================================================
//                  CAPTURE IMAGE
// ========================================================

void handleCapture() {

  camera_fb_t *fb = esp_camera_fb_get();

  if (!fb) {

    Serial.println("Camera capture failed");

    server.send(
      500,
      "text/plain",
      "Camera capture failed"
    );

    return;
  }


  uint8_t *jpg_buf = NULL;
  size_t jpg_len = 0;


  // GC2145 is running RGB565.
  // Convert RGB565 frame to JPEG.

  bool converted = frame2jpg(
    fb,
    80,
    &jpg_buf,
    &jpg_len
  );


  esp_camera_fb_return(fb);


  if (!converted) {

    Serial.println("JPEG conversion failed");

    server.send(
      500,
      "text/plain",
      "JPEG conversion failed"
    );

    return;
  }


  server.setContentLength(jpg_len);

  server.send(
    200,
    "image/jpeg",
    ""
  );


  WiFiClient client = server.client();

  client.write(
    jpg_buf,
    jpg_len
  );


  free(jpg_buf);
}
```

```
// ========================================================
//                      SETUP
// ========================================================

void setup() {

  Serial.begin(115200);

  delay(2000);


  Serial.println();
  Serial.println("================================");
  Serial.println(" ESP32S-CAM GC2145 WEB CAMERA");
  Serial.println("================================");


  // ================================================
  // Camera configuration
  // ================================================

  camera_config_t config;

  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer   = LEDC_TIMER_0;


  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;


  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;

  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href  = HREF_GPIO_NUM;


  config.pin_sccb_sda = SIOD_GPIO_NUM;
  config.pin_sccb_scl = SIOC_GPIO_NUM;


  config.pin_pwdn  = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;


  // GC2145 conservative clock

  config.xclk_freq_hz = 10000000;


  // IMPORTANT:
  // GC2145 does not use native JPEG

  config.pixel_format = PIXFORMAT_RGB565;


  // Start with QVGA
  // 320 x 240

  config.frame_size = FRAMESIZE_QVGA;


  config.grab_mode =
    CAMERA_GRAB_WHEN_EMPTY;


  config.fb_location =
    CAMERA_FB_IN_PSRAM;
```

```
config.jpeg_quality = 12;

config.fb_count = 1;


// ================================================
// Initialize Camera
// ================================================

Serial.println("Initializing GC2145 camera...");


esp_err_t err =
  esp_camera_init(&config);


if (err != ESP_OK) {

  Serial.printf(
    "Camera init failed: 0x%x\n",
    err
  );

  return;
}


Serial.println(
  "Camera initialized successfully!"
);


// ================================================
// Check Camera Sensor
// ================================================

sensor_t *sensor =
  esp_camera_sensor_get();


if (sensor != NULL) {

  Serial.printf(
    "Camera Sensor PID: 0x%04X\n",
    sensor->id.PID
  );

}


// ================================================
// Create ESP32 Wi-Fi Hotspot
// ================================================

Serial.println();
Serial.println("Starting Wi-Fi hotspot...");


WiFi.mode(WIFI_AP);


WiFi.softAP(
  ap_ssid,
  ap_password
);


delay(1000);


IPAddress IP =
  WiFi.softAPIP();
```

```
  Serial.println();
  Serial.println("================================");

  Serial.println("Wi-Fi Hotspot Started!");

  Serial.print("Wi-Fi Name: ");
  Serial.println(ap_ssid);

  Serial.print("Password: ");
  Serial.println(ap_password);

  Serial.print("Camera URL: http://");
  Serial.println(IP);

  Serial.println("================================");


  // ================================================
  // Web Server
  // ================================================

  server.on(
    "/",
    HTTP_GET,
    handleRoot
  );


  server.on(
    "/capture",
    HTTP_GET,
    handleCapture
  );


  server.begin();


  Serial.println();
  Serial.println("Camera Web Server Started!");

}


// ================================================
//                      LOOP
// ================================================

void loop() {

  server.handleClient();

  delay(2);

}
```

Upload করার পর

Upload successful হলে **`Serial Monitor → 115200 baud`** খুলে ESP32S-CAM-এর **RST** চাপো।

এমন output পাওয়া উচিত:

```
================================
 ESP32S-CAM GC2145 WEB CAMERA
================================

Initializing GC2145 camera...

Camera initialized successfully!
Camera Sensor PID: 0x2145
```

```
Starting Wi-Fi hotspot...

================================
Wi-Fi Hotspot Started!
Wi-Fi Name: ESP32-GC2145-CAM
Password: 12345678
Camera URL: http://192.168.4.1
================================

Camera Web Server Started!
```

তারপর phone/Mac-এর Wi-Fi থেকে:

```
ESP32-GC2145-CAM
```

connect করো।

Password:

```
12345678
```

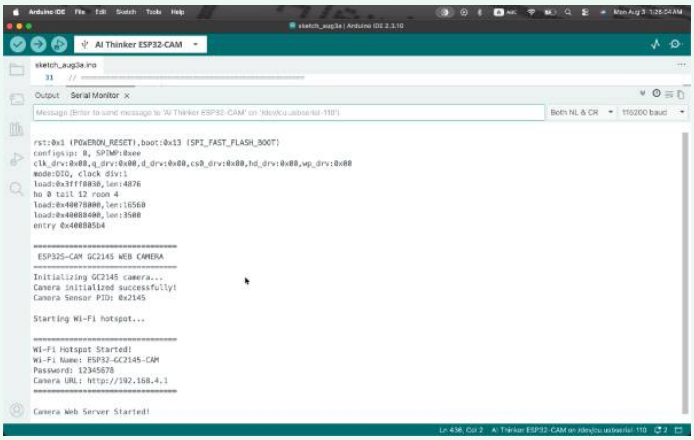
তারপর browser-এ `http://192.168.4.1` লিখে ঢুকবে।

এটা internet ছাড়াই হবে:

```
ESP32S-CAM
      │
      │ creates Wi-Fi
      ▼
ESP32-GC2145-CAM
      │
      │ Wi-Fi
      ▼
 Phone / Mac
      │
      ▼
192.168.4.1
      │
      ▼
GC2145 Camera View
```

প্রথম test-এ QVGA (320×240) রেখেছি। কারণ GC2145 থেকে RGB565 frame নিয়ে ESP32-কে JPEG conversion করতে হচ্ছে; সরাসরি high resolution দিয়ে শুরু করলে memory/performance সমস্যা হতে পারে।

Code upload করে Serial Monitor-এর output পাঠাও। `PID: 0x2145` এবং `Camera Web Server Started!` এলে তারপর আমরা **actual live stream (MJPEG)** এবং resolution/quality controls যোগ করতে পারি।



Perfect! 🎉 Screenshot অনুযায়ী পুরো system এখন ঠিকমতো চলছে।

```
Camera initialized successfully! ✅
Camera Sensor PID: 0x2145      ✅

Wi-Fi Hotspot Started!         ✅
Wi-Fi Name: ESP32-GC2145-CAM
Password: 12345678

Camera URL: http://192.168.4.1

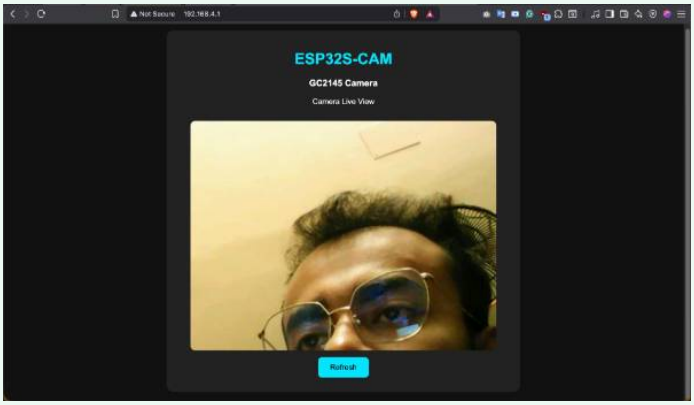
Camera Web Server Started!     ✅
```

এখন ESP32 থেকে USB খুলবে না। এরপর:

1. Phone/Mac-এর **Wi-Fi Settings** খোলো।
2. **`ESP32-GC2145-CAM`**-এ connect করো।
3. Password দাও: **`12345678`**
4. **"No Internet Connection" দেখালে চিন্তার কিছু নেই**—ESP32 নিজেই local Wi-Fi network বানিয়েছে।
5. Safari/Chrome খুলে address bar-এ **`http://192.168.4.1`** লিখে Enter দাও।

তখন camera page আসার কথা এবং প্রতি ১ সেকেন্ডে নতুন image refresh হবে।

এখন `192.168.4.1` open করে কী দেখাচ্ছে তার screenshot পাঠাও। যদি image না আসে/ভাঙা image icon আসে, তাহলে **`/capture`** JPEG conversion অংশটা আমরা ঠিক করব।



Yes! 🎉 এটা এখন **পুরোপুরি কাজ করছে**।

Screenshot থেকে দেখা যাচ্ছে:

- **`192.168.4.1`** web server চলছে ✅
- GC2145 image capture করছে ✅
- RGB565 → JPEG conversion কাজ করছে ✅
- Browser image receive করছে ✅
- ESP32 hotspot কাজ করছে ✅
- Auto-refresh system-ও চলছে ✅

তবে এটা এখন technically **live video stream না**—প্রতি ১ সেকেন্ডে নতুন ছবি **`/capture`** থেকে load হচ্ছে।

পরের ধাপে আমরা এটাকে **proper live camera** করতে পারি:

```
GC2145
   ↓
RGB565 Capture
   ↓
JPEG Conversion
   ↓
MJPEG Stream
   ↓
Wi-Fi
   ↓
Browser
   ↓
LIVE VIDEO 📹
```

সাথে web page-এ **Live Stream / Stop Stream / Capture Photo / Resolution / Image Quality / Brightness / Contrast / Flip / Mirror** control-ও দিতে পারি।

তোমার বর্তমান setup stable হওয়ায় আমি পরের code-টা এই working configuration-এর উপরেই বানাব, যাতে **`GC2145 PID 0x2145`** support নষ্ট না হয়।

> ok do it

চলো, এবার তোমার **working ESP32S-CAM + GC2145** setup-কে proper camera dashboard বানাই। নিচের code-এ থাকবে **MJPEG Live Stream, Stop/Start, Capture Photo, resolution, JPEG quality, brightness, contrast, mirror এবং flip**।

পুরনো code পুরো replace করে এটা দাও:

```cpp
#include <WiFi.h>
#include "esp_camera.h"
#include "img_converters.h"
#include "esp_http_server.h"

// ================================================
// ESP32S-CAM + GC2145 PIN CONFIG
// Same pin mapping that already worked for your camera
// ================================================

#define PWDN_GPIO_NUM     32
#define RESET_GPIO_NUM    -1

#define XCLK_GPIO_NUM      0
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27

#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5

#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22

// ================================================
// Wi-Fi Access Point
// ================================================

const char* AP_SSID = "ESP32-GC2145-CAM";
```

```
const char* AP_PASS = "12345678";

httpd_handle_t camera_httpd = NULL;


// ================================================
// Web Page
// ================================================

static const char PROGMEM INDEX_HTML[] = R"rawliteral(
<!DOCTYPE html>
<html>

<head>

<meta charset="UTF-8">

<meta
 name="viewport"
 content="width=device-width, initial-scale=1.0">

<title>ESP32S-CAM GC2145</title>

<style>

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: #0d1117;
  color: #ffffff;
  font-family: Arial, Helvetica, sans-serif;
  text-align: center;
}

header {
  padding: 18px;
  background: #161b22;
}

h1 {
  margin: 0;
  color: #20d9ff;
  font-size: 28px;
}

.subtitle {
  color: #aaa;
  margin-top: 6px;
}

.container {
  max-width: 800px;
  margin: auto;
  padding: 20px;
}

.camera-box {
  background: #161b22;
  padding: 15px;
  border-radius: 16px;
}

#stream {
  display: block;
  width: 100%;
  max-width: 640px;
  margin: auto;
  border-radius: 12px;
  background: #000;
  min-height: 240px;
  object-fit: contain;
}
```

```
.controls {
  margin-top: 15px;
}

button {
  border: none;
  padding: 11px 18px;
  margin: 5px;
  border-radius: 8px;
  font-size: 15px;
  cursor: pointer;
}

.start {
  background: #20d9ff;
}

.stop {
  background: #ff5252;
  color: white;
}

.capture {
  background: #4caf50;
  color: white;
}

.panel {
  margin-top: 20px;
  padding: 18px;
  background: #161b22;
  border-radius: 16px;
  text-align: left;
}

.panel h2 {
  color: #20d9ff;
  text-align: center;
}

.control-row {
  margin: 16px 0;
}

label {
  display: block;
  margin-bottom: 7px;
}

select {
  width: 100%;
  padding: 10px;
  border-radius: 7px;
  border: none;
}

input[type="range"] {
  width: 100%;
}

.switch-row {
  display: flex;
  justify-content: space-between;
  margin: 15px 0;
}

.status {
  margin-top: 15px;
  color: #8b949e;
}

</style>

</head>
```

```
<body>

<header>

<h1>ESP32S-CAM</h1>

<div class="subtitle">
GC2145 Live Camera
</div>

</header>


<div class="container">


<div class="camera-box">

<img id="stream">

<div class="controls">

<button
 class="start"
 onclick="startStream()">
▶ Start Stream
</button>

<button
 class="stop"
 onclick="stopStream()">
■ Stop
</button>

<button
 class="capture"
 onclick="capturePhoto()">
📷 Capture
</button>

</div>

<div class="status" id="status">
Camera Ready
</div>

</div>


<div class="panel">

<h2>Camera Controls</h2>


<div class="control-row">

<label>Resolution</label>

<select
 id="resolution"
 onchange="setResolution(this.value)">

<option value="1">
QQVGA 160 × 120
</option>

<option value="2" selected>
QVGA 320 × 240
</option>

<option value="5">
VGA 640 × 480
</option>
```

```
</select>

</div>


<div class="control-row">

<label>
JPEG Quality:
<span id="qualityValue">75</span>
</label>

<input
 type="range"
 min="30"
 max="90"
 value="75"
 id="quality"
 oninput="
 document.getElementById('qualityValue').innerText=this.value
 "
 onchange="setControl('quality',this.value)">
</div>


<div class="control-row">

<label>
Brightness:
<span id="brightnessValue">0</span>
</label>

<input
 type="range"
 min="-2"
 max="2"
 value="0"
 oninput="
 document.getElementById('brightnessValue').innerText=this.value
 "
 onchange="setControl('brightness',this.value)">
</div>


<div class="control-row">

<label>
Contrast:
<span id="contrastValue">0</span>
</label>

<input
 type="range"
 min="-2"
 max="2"
 value="0"
 oninput="
 document.getElementById('contrastValue').innerText=this.value
 "
 onchange="setControl('contrast',this.value)">
</div>


<div class="switch-row">

<label>Mirror</label>

<input
 type="checkbox"
 onchange="
 setControl('mirror',this.checked ? 1 : 0)
 ">

</div>
```

```
<div class="switch-row">

<label>Vertical Flip</label>

<input
 type="checkbox"
 onchange="
 setControl('flip',this.checked ? 1 : 0)
 ">

</div>


</div>

</div>


<script>

const stream =
 document.getElementById("stream");

const statusText =
 document.getElementById("status");


function startStream() {

  stream.src =
   "/stream?t=" + Date.now();

  statusText.innerText =
   "Live Stream Running";

}


function stopStream() {

  stream.src = "";

  statusText.innerText =
   "Stream Stopped";

}


function capturePhoto() {

  window.open(
    "/capture?t=" + Date.now(),
    "_blank"
  );

}


function setResolution(value) {

  fetch(
    "/control?var=framesize&val=" + value
  );

}


function setControl(variable,value) {

  fetch(
    "/control?var=" +
    variable +
    "&val=" +
    value
```

```
  );

}

window.onload = startStream;

</script>


</body>

</html>
)rawliteral";


// ==================================================
// Home Page
// ==================================================

static esp_err_t index_handler(
  httpd_req_t *req
) {

  httpd_resp_set_type(
    req,
    "text/html"
  );

  return httpd_resp_send(
    req,
    INDEX_HTML,
    HTTPD_RESP_USE_STRLEN
  );
}


// ==================================================
// Capture Single JPEG
// ==================================================

static esp_err_t capture_handler(
  httpd_req_t *req
) {

  camera_fb_t *fb =
    esp_camera_fb_get();

  if (!fb) {

    Serial.println(
      "Camera capture failed"
    );

    httpd_resp_send_500(req);

    return ESP_FAIL;
  }


  uint8_t *jpg_buf = NULL;
  size_t jpg_len = 0;


  bool ok = frame2jpg(
    fb,
    75,
    &jpg_buf,
    &jpg_len
  );


  esp_camera_fb_return(fb);


  if (!ok) {
```

```
    Serial.println(
      "JPEG conversion failed"
    );

    httpd_resp_send_500(req);

    return ESP_FAIL;
  }


  httpd_resp_set_type(
    req,
    "image/jpeg"
  );


  httpd_resp_set_hdr(
    req,
    "Content-Disposition",
    "inline; filename=GC2145.jpg"
  );


  esp_err_t result =
    httpd_resp_send(
      req,
      (const char *)jpg_buf,
      jpg_len
    );


  free(jpg_buf);


  return result;
}


// ================================================
// MJPEG STREAM
// ================================================

static esp_err_t stream_handler(
  httpd_req_t *req
) {

  esp_err_t res = ESP_OK;

  camera_fb_t *fb = NULL;

  uint8_t *jpg_buf = NULL;

  size_t jpg_len = 0;


  static const char *STREAM_TYPE =
    "multipart/x-mixed-replace;"
    "boundary=frame";


  static const char *STREAM_BOUNDARY =
    "\r\n--frame\r\n";


  static const char *STREAM_PART =
    "Content-Type: image/jpeg\r\n"
    "Content-Length: %u\r\n\r\n";


  httpd_resp_set_type(
    req,
    STREAM_TYPE
  );
```

```
  char part_buf[64];


  while (true) {

    fb = esp_camera_fb_get();


    if (!fb) {

      Serial.println(
        "Camera capture failed"
      );

      res = ESP_FAIL;

      break;
    }


    jpg_buf = NULL;
    jpg_len = 0;


    // GC2145 frame is RGB565.
    // Convert each frame to JPEG.

    bool converted = frame2jpg(
      fb,
      70,
      &jpg_buf,
      &jpg_len
    );


    esp_camera_fb_return(fb);

    fb = NULL;


    if (!converted) {

      Serial.println(
        "Frame JPEG conversion failed"
      );

      res = ESP_FAIL;

      break;
    }


    res = httpd_resp_send_chunk(
      req,
      STREAM_BOUNDARY,
      strlen(STREAM_BOUNDARY)
    );


    if (res != ESP_OK) {

      free(jpg_buf);

      break;
    }


    size_t header_len =
      snprintf(
        part_buf,
        sizeof(part_buf),
        STREAM_PART,
        jpg_len
      );
```

```
    res = httpd_resp_send_chunk(
      req,
      part_buf,
      header_len
    );


    if (res != ESP_OK) {

      free(jpg_buf);

      break;
    }


    res = httpd_resp_send_chunk(
      req,
      (const char *)jpg_buf,
      jpg_len
    );


    free(jpg_buf);

    jpg_buf = NULL;


    if (res != ESP_OK) {

      break;

    }


    // Reduce CPU load a little.
    delay(40);

  }


  return res;
}


// ================================================
// Camera Control
// ================================================

static esp_err_t control_handler(
  httpd_req_t *req
) {

  char query[128];

  char variable[32];

  char value[32];


  if (
    httpd_req_get_url_query_str(
      req,
      query,
      sizeof(query)
    ) != ESP_OK
  ) {

    httpd_resp_send_404(req);

    return ESP_FAIL;
  }
```

```
  if (
    httpd_query_key_value(
      query,
      "var",
      variable,
      sizeof(variable)
    ) != ESP_OK
  ) {

    httpd_resp_send_404(req);

    return ESP_FAIL;
  }


  if (
    httpd_query_key_value(
      query,
      "val",
      value,
      sizeof(value)
    ) != ESP_OK
  ) {

    httpd_resp_send_404(req);

    return ESP_FAIL;
  }


  int val = atoi(value);


  sensor_t *s =
    esp_camera_sensor_get();


  if (!s) {

    httpd_resp_send_500(req);

    return ESP_FAIL;
  }


  int result = 0;


  if (!strcmp(variable, "framesize")) {

    framesize_t fs;


    if (val == 1) {

      fs = FRAMESIZE_QQVGA;

    }
    else if (val == 5) {

      fs = FRAMESIZE_VGA;

    }
    else {

      fs = FRAMESIZE_QVGA;

    }


    result =
      s->set_framesize(
        s,
        fs
      );
```

```
  }


  else if (
    !strcmp(
      variable,
      "brightness"
    )
  ) {

    result =
      s->set_brightness(
        s,
        val
      );

  }


  else if (
    !strcmp(
      variable,
      "contrast"
    )
  ) {

    result =
      s->set_contrast(
        s,
        val
      );

  }


  else if (
    !strcmp(
      variable,
      "mirror"
    )
  ) {

    result =
      s->set_hmirror(
        s,
        val
      );

  }


  else if (
    !strcmp(
      variable,
      "flip"
    )
  ) {

    result =
      s->set_vflip(
        s,
        val
      );

  }


  // quality is handled during
  // software JPEG conversion.
  else if (
    !strcmp(
      variable,
      "quality"
    )
```

```
  ) {

    result = 0;

  }


  else {

    httpd_resp_send_404(req);

    return ESP_FAIL;
  }


  if (result != 0) {

    httpd_resp_send_500(req);

    return ESP_FAIL;
  }


  httpd_resp_set_hdr(
    req,
    "Access-Control-Allow-Origin",
    "*"
  );


  return httpd_resp_send(
    req,
    NULL,
    0
  );
}


// ================================================
// Start Web Server
// ================================================

void startCameraServer() {

  httpd_config_t config =
    HTTPD_DEFAULT_CONFIG();


  config.server_port = 80;

  config.max_uri_handlers = 8;

  config.stack_size = 8192;


  // ----------------------
  // HOME
  // ----------------------

  httpd_uri_t index_uri = {
    .uri = "/",
    .method = HTTP_GET,
    .handler = index_handler,
    .user_ctx = NULL
  };


  // ----------------------
  // CAPTURE
  // ----------------------

  httpd_uri_t capture_uri = {
    .uri = "/capture",
    .method = HTTP_GET,
    .handler = capture_handler,
```

```
    .user_ctx = NULL
  };


  // ---------------------
  // STREAM
  // ---------------------

  httpd_uri_t stream_uri = {
    .uri = "/stream",
    .method = HTTP_GET,
    .handler = stream_handler,
    .user_ctx = NULL
  };


  // ---------------------
  // CONTROL
  // ---------------------

  httpd_uri_t control_uri = {
    .uri = "/control",
    .method = HTTP_GET,
    .handler = control_handler,
    .user_ctx = NULL
  };


  Serial.println(
    "Starting HTTP server..."
  );


  if (
    httpd_start(
      &camera_httpd,
      &config
    ) == ESP_OK
  ) {

    httpd_register_uri_handler(
      camera_httpd,
      &index_uri
    );


    httpd_register_uri_handler(
      camera_httpd,
      &capture_uri
    );


    httpd_register_uri_handler(
      camera_httpd,
      &stream_uri
    );


    httpd_register_uri_handler(
      camera_httpd,
      &control_uri
    );


    Serial.println(
      "HTTP server started!"
    );

  }

}


// ================================================
// SETUP
```

```
// ======================================================

void setup() {

  Serial.begin(115200);

  delay(2000);


  Serial.println();
  Serial.println(
    "================================="
  );

  Serial.println(
    " ESP32S-CAM GC2145 LIVE STREAM"
  );

  Serial.println(
    "================================="
  );


  // ======================================================
  // Camera configuration
  // ======================================================

  camera_config_t config;


  config.ledc_channel =
    LEDC_CHANNEL_0;

  config.ledc_timer =
    LEDC_TIMER_0;


  config.pin_d0 =
    Y2_GPIO_NUM;

  config.pin_d1 =
    Y3_GPIO_NUM;

  config.pin_d2 =
    Y4_GPIO_NUM;

  config.pin_d3 =
    Y5_GPIO_NUM;

  config.pin_d4 =
    Y6_GPIO_NUM;

  config.pin_d5 =
    Y7_GPIO_NUM;

  config.pin_d6 =
    Y8_GPIO_NUM;

  config.pin_d7 =
    Y9_GPIO_NUM;


  config.pin_xclk =
    XCLK_GPIO_NUM;

  config.pin_pclk =
    PCLK_GPIO_NUM;

  config.pin_vsync =
    VSYNC_GPIO_NUM;

  config.pin_href =
    HREF_GPIO_NUM;
```

```
  config.pin_sccb_sda =
    SIOD_GPIO_NUM;

  config.pin_sccb_scl =
    SIOC_GPIO_NUM;


  config.pin_pwdn =
    PWDN_GPIO_NUM;

  config.pin_reset =
    RESET_GPIO_NUM;


  // Same stable GC2145 setting
  // that worked previously.

  config.xclk_freq_hz =
    10000000;


  // IMPORTANT:
  // GC2145 does not output
  // native JPEG.

  config.pixel_format =
    PIXFORMAT_RGB565;


  // Start at 320 x 240

  config.frame_size =
    FRAMESIZE_QVGA;


  config.grab_mode =
    CAMERA_GRAB_WHEN_EMPTY;


  config.fb_location =
    CAMERA_FB_IN_PSRAM;


  config.jpeg_quality = 12;

  config.fb_count = 1;


  // ================================================
  // Initialize Camera
  // ================================================

  Serial.println(
    "Initializing GC2145..."
  );


  esp_err_t err =
    esp_camera_init(
      &config
    );


  if (err != ESP_OK) {

    Serial.printf(
      "Camera initialization failed: 0x%x\n",
      err
    );

    return;

  }
```

```
  Serial.println(
    "Camera initialized successfully!"
  );


  sensor_t *sensor =
    esp_camera_sensor_get();


  if (sensor) {

    Serial.printf(
      "Sensor PID: 0x%04X\n",
      sensor->id.PID
    );

  }


  // ================================================
  // Wi-Fi AP
  // ================================================

  Serial.println();
  Serial.println(
    "Starting Wi-Fi hotspot..."
  );


  WiFi.mode(WIFI_AP);


  bool apStarted =
    WiFi.softAP(
      AP_SSID,
      AP_PASS
    );


  if (!apStarted) {

    Serial.println(
      "Failed to start Wi-Fi AP!"
    );

    return;

  }


  delay(1000);


  IPAddress ip =
    WiFi.softAPIP();


  Serial.println();
  Serial.println(
    "================================="
  );

  Serial.println(
    "Wi-Fi Hotspot Started!"
  );


  Serial.print(
    "Wi-Fi Name: "
  );

  Serial.println(
    AP_SSID
  );
```

```
  Serial.print(
    "Password: "
  );

  Serial.println(
    AP_PASS
  );


  Serial.print(
    "Camera URL: http://"
  );

  Serial.println(
    ip
  );


  Serial.println(
    "================================"
  );


  // ================================================
  // Start HTTP server
  // ================================================

  startCameraServer();


  Serial.println();
  Serial.println(
    "GC2145 Camera Ready!"
  );

}


// ================================================
// LOOP
// ================================================

void loop() {

  delay(1000);

}
```

Upload এবং test

আগের মতোই:

```
Board: AI Thinker ESP32-CAM
Port: /dev/cu.usbserial-110
Upload Speed: 115200
Partition: Huge APP
```

Upload শেষে RST চাপবে। Serial Monitor **115200**-এ ideally দেখাবে:

```
ESP32S-CAM GC2145 LIVE STREAM

Initializing GC2145...
Camera initialized successfully!
Sensor PID: 0x2145

Wi-Fi Hotspot Started!
Wi-Fi Name: ESP32-GC2145-CAM
Password: 12345678
Camera URL: http://192.168.4.1
```

```
Starting HTTP server...
HTTP server started!

GC2145 Camera Ready!
```

তারপর **`ESP32-GC2145-CAM`** Wi-Fi-তে connect করে browser-এ **`http://192.168.4.1`** open করবে।

একটা গুরুত্বপূর্ণ বিষয়: **GC2145 → RGB565 → software JPEG conversion** হচ্ছে। তাই OV2640-এর মতো খুব high FPS আশা করা ঠিক হবে না। প্রথমে **QVGA 320×240**-তেই test করো। VGA 640×480 দিলে ESP32-এর CPU/memory load অনেক বেড়ে যেতে পারে।

আর এই version-এ Quality slider-এর UI আছে, কিন্তু stream conversion বর্তমানে fixed **`70`** এবং capture fixed **`75`**; চাইলে পরের revision-এ slider-টাকে actual conversion quality variable-এর সাথে connect করে দেব।

ChatGPT can make mistakes. Check important info.